GShard

Danyang Zhuo

Duke University

Challenges for scaling

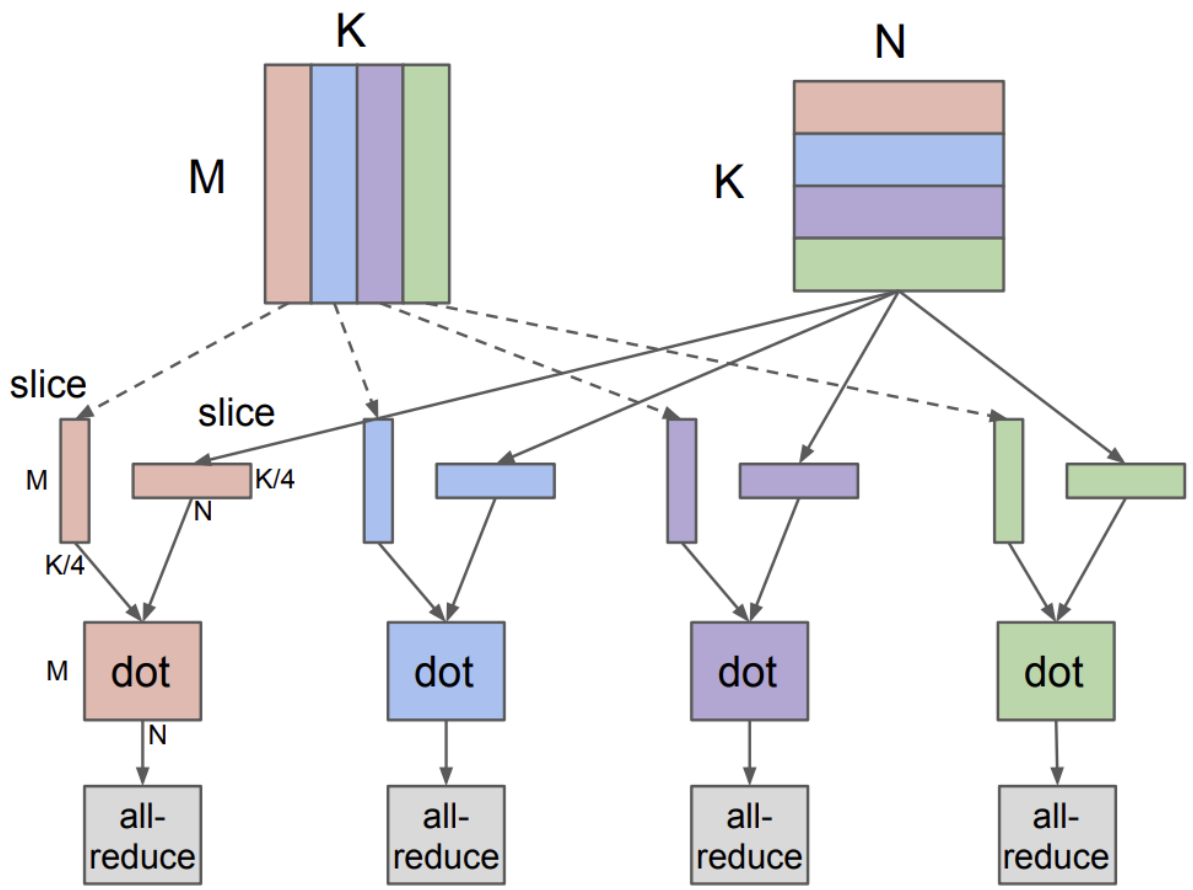
- Architecture-specific model parallelism support
- Super-linear scaling of computation cost vs model size
- Infrastructure scalability for giant model representation
- Non-trivial efforts for implementing partitioning strategies

Techniques

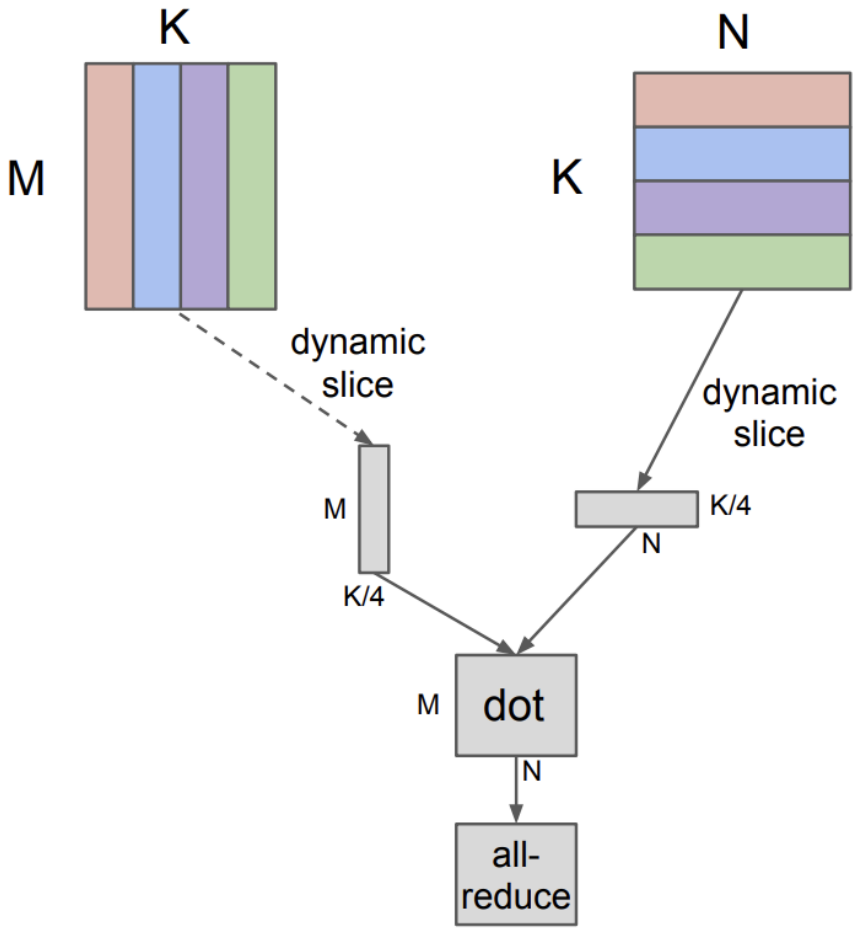
- MoE for sub-linear scaling

- The Power of Abstraction
  - User annotates a few critical tensors in the model with partitioning policies. XLA compiles for automated parallelization.

- Scalable Compilers
  - XLA + SPMD (Single Program Multiple Data)



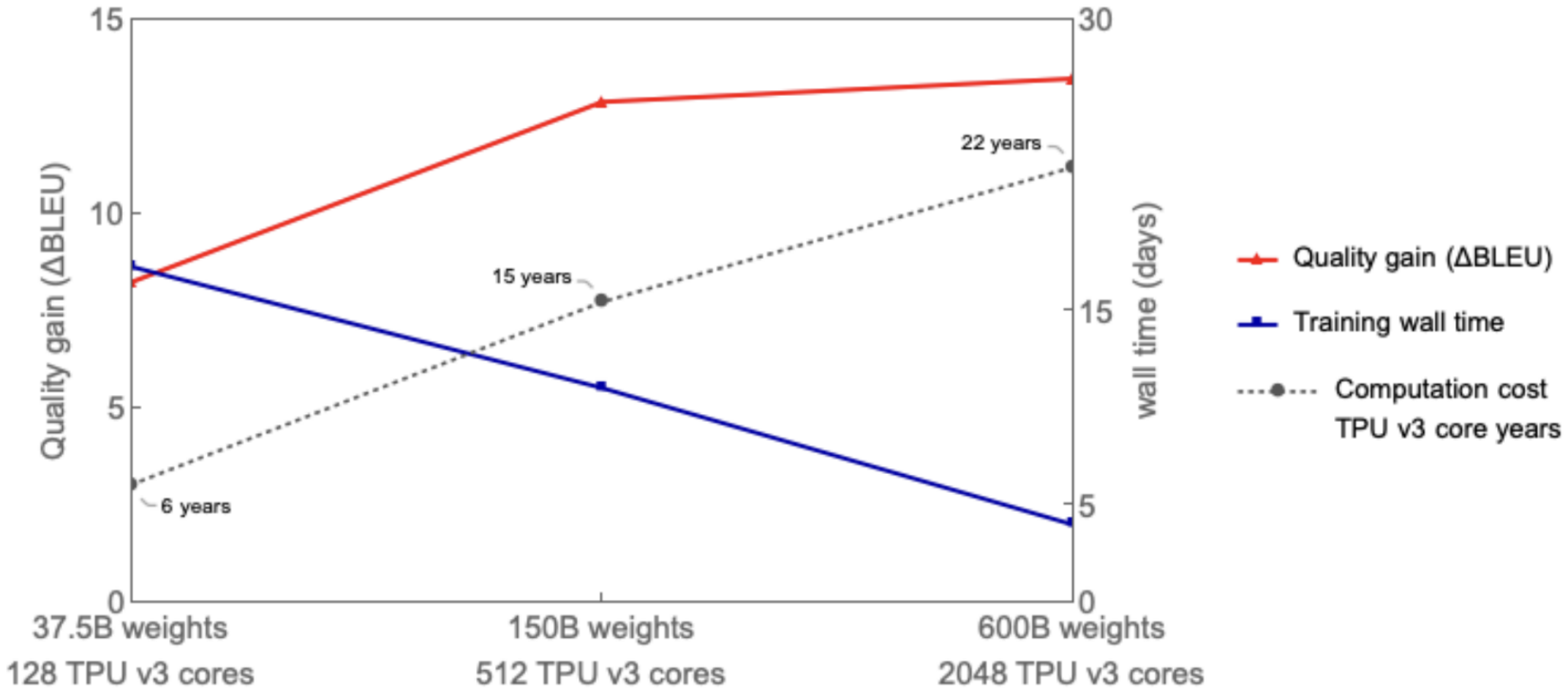
(a) MPMD Partition



(b) SPMD Partition

What’s the difference between XLA and MPI?

- MPI
  - Collective communication + sharding are explicitly programmed
- XLA
  - Users only provide annotations. Collective communication is inserted by the compiler.


15
10
5
0
30
15
5
0
Quality gain (ΔBLEU)
wall time (days)
22 years
15 years
6 years
Quality gain (ΔBLEU)
Training wall time
Computation cost
TPU v3 core years
37.5B weights
128 TPU v3 cores
150B weights
512 TPU v3 cores
600B weights
2048 TPU v3 cores

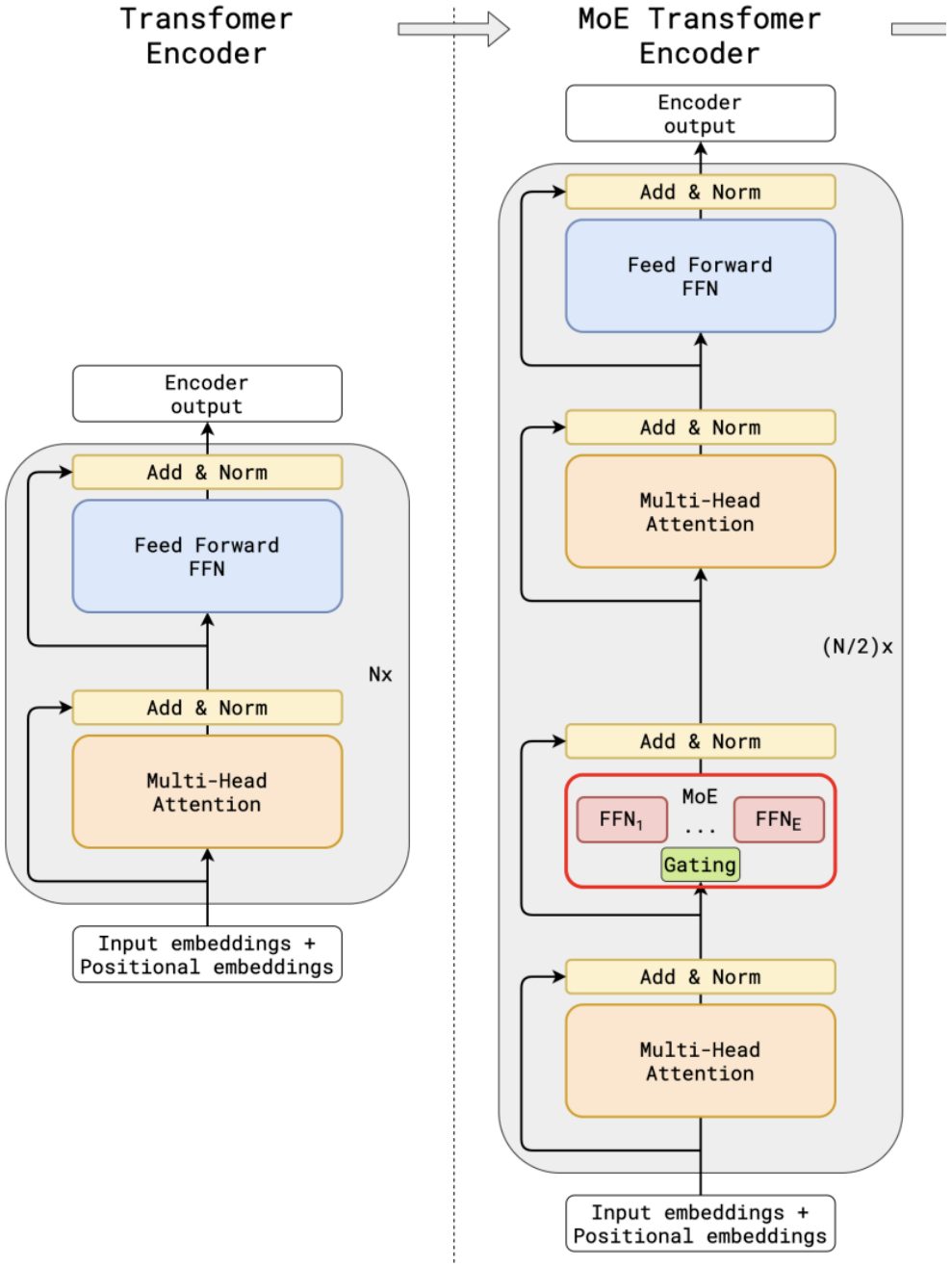
The Mixture-of-Experts (MoE) layer used in our model is based on [16] with variations in the sparse gating function and the auxiliary loss being used. A MoE layer for Transformer consists of E feed-forward networks $\text{FFN}_1 \ldots \text{FFN}_E$:

$$\mathcal{G}_{s,E} = \text{GATE}(x_s) \tag{1}$$

$$\text{FFN}_e(x_s) = wo_e \cdot \text{ReLU}(wi_e \cdot x_s) \tag{2}$$

$$y_s = \sum_{e=1}^{E} \mathcal{G}_{s,e} \cdot \text{FFN}_e(x_s) \tag{3}$$

where x_s is the input token to the MoE layer, wiand wobeing the input and output projection matrices for the feed-forward layer (an expert). Vector $\mathcal{G}_{s,E}$ is computed by a gating network. $\mathcal{G}_{s,E}$ has one non-negative for each expert, most of which are zeros meaning the token is not dispatched to that expert. The token is dispatched to a very small number of experts. We choose to let each token dispatched to at most two experts. The corresponding entries in $\mathcal{G}_{s,E}$ are non-zeros, representing how much an expert contributes to the final network output. Every expert FFN_e applies to x_s a fully-connected 2-layer network using ReLU [29] activation function. The output of the MoE layer, y_s, is the weighted average of outputs from all the selected experts.

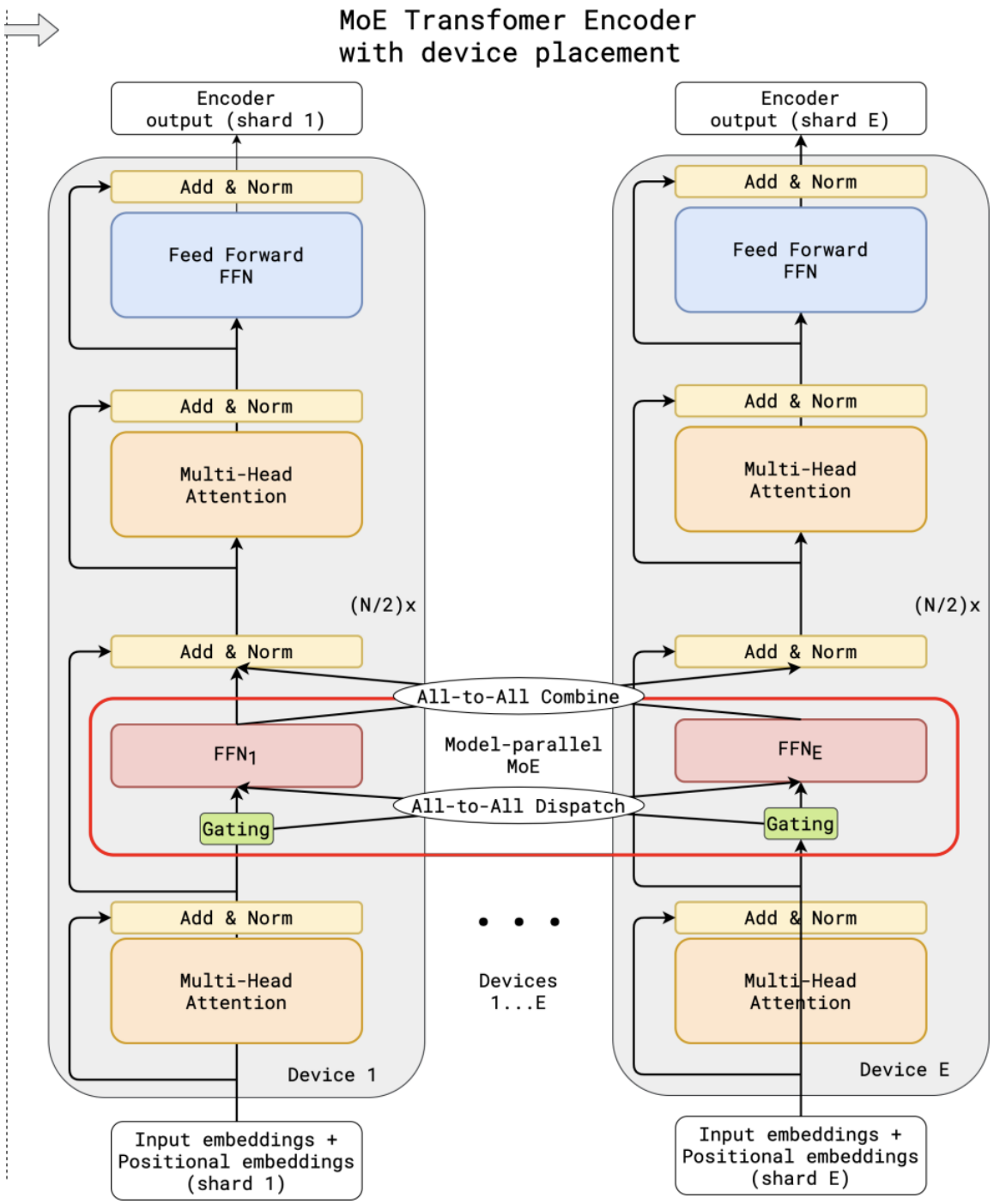

Transfomer Encoder
MoE Transfomer Encoder
Encoder output
Add & Norm
Feed Forward FFN
Add & Norm
Multi-Head Attention
Nx
Input embeddings + Positional embeddings
Encoder output
Add & Norm
Feed Forward FFN
Add & Norm
Multi-Head Attention
(N/2)x
Add & Norm
MoE
FFN1
...
FFNE
Gating
Add & Norm
Multi-Head Attention
Input embeddings + Positional embeddings

- **Balanced load** It is desirable that the MoE layer to sparsely activate the experts for a given token. A naive solution would be just to choose the top-k experts according to the softmax probability distribution. However, it is known that this approach leads to load imbalance problem for training [16]: most tokens seen during training would have been dispatched to a small number of experts, amassing a very large input buffer for only a few (busy) experts leaving other experts untrained, slowing down the training. Meanwhile many other experts do not get sufficiently trained at all. A better design of the gating function would distribute processing burden more evenly across all experts.
- **Efficiency at scale** It would be rather trivial to achieve a balanced load if the gating function is done sequentially. The computation cost for the gating function alone is at least $O(NE)$ for all N tokens in the input batch given E experts. However, in our study, N is in the order of millions and E is in the order of thousands, a sequential implementation of the gating function would keep most of the computational resources idle most of the time. Therefore, we need an efficient parallel implementation of the gating function to leverage many devices.

Algorithm 1: Group-level top-2 gating with auxiliary loss

Data: x_S, a group of tokens of size S
Data: C, Expert capacity allocated to this group
Result: $\mathcal{G}_{S,E}$, group combine weights
Result: ℓ_{aux}, group auxiliary loss

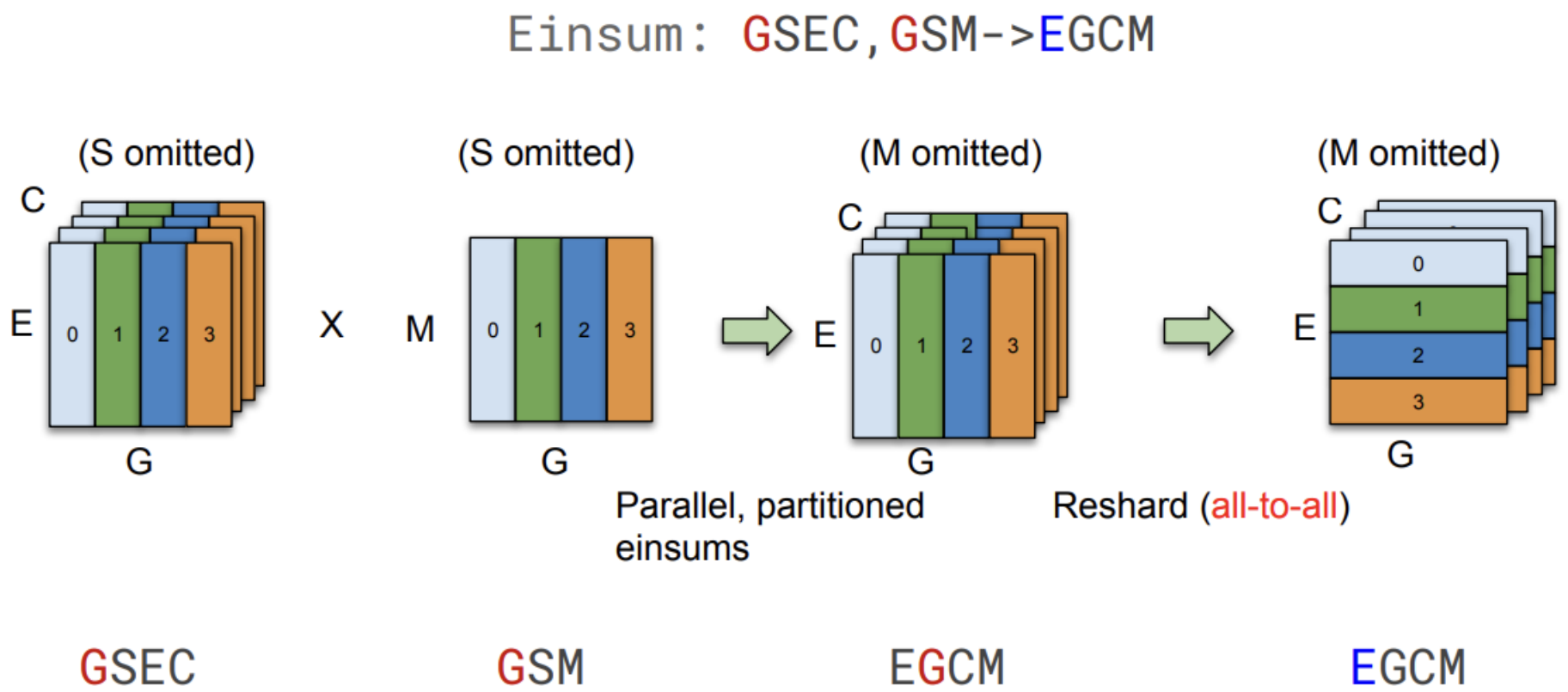
$c_E \leftarrow 0$ ▷ gating decisions per expert
$g_{S,E} \leftarrow softmax(wg \cdot x_S)$ ▷ gates per token per expert, wg are trainable weights
$m_E \leftarrow \frac{1}{S}\sum_{s=1}^{s} g_{s,E}$ ▷ mean gates per expert
for $s \leftarrow 1$ **to** S **do**
  $g1, e1, g2, e2 = top_2(g_{s,E})$ ▷ top-2 gates and expert indices
  $g1 \leftarrow g1/(g1 + g2)$ ▷ normalized $g1$
  $c \leftarrow c_{e1}$ ▷ position in $e1$ expert buffer
  if $c_{e1} < C$ **then**
    $\mathcal{G}_{s,e1} \leftarrow g1$ ▷ $e1$ expert combine weight for x_s
  end
  $c_{e1} \leftarrow c + 1$ ▷ incrementing $e1$ expert decisions count
end
$\ell_{aux} = \frac{1}{E}\sum_{e=1}^{E} \frac{c_e}{S} \cdot m_e$
for $s \leftarrow 1$ **to** S **do**
  $g1, e1, g2, e2 = top_2(g_{s,E})$ ▷ top-2 gates and expert indices
  $g2 \leftarrow g2/(g1 + g2)$ ▷ normalized $g2$
  $rnd \leftarrow uniform(0, 1)$ ▷ dispatch to second-best expert with probability $\propto 2 \cdot g2$
  $c \leftarrow c_{e2}$ ▷ position in $e2$ expert buffer
  if $c < C \wedge 2 \cdot g2 > rnd$ **then**
    $\mathcal{G}_{s,e2} \leftarrow g2$ ▷ $e2$ expert combine weight for x_s
  end
  $c_{e2} \leftarrow c + 1$
end


MoE Transfomer Encoder
with device placement
Encoder
output (shard 1)
Encoder
output (shard E)
Add & Norm
Feed Forward
FFN
Add & Norm
Multi-Head
Attention
(N/2)x
Add & Norm
All-to-All Combine
FFN1
Model-parallel
MoE
FFNE
All-to-All Dispatch
Gating
Gating
Add & Norm
Multi-Head
Attention
Devices
1...E
Device 1
Device E
Input embeddings +
Positional embeddings
(shard 1)
Input embeddings +
Positional embeddings
(shard E)

`Top2Gating` in Algorithm 2 computes the union of all group-local $\mathcal{G}_{S,E}$ described in Algorithm 1. `combine_weights` is a 4-D tensor with shape `[G, S, E, C]`. The value `combine_weights[g, s, e, c]` is non-zero when the input token s in group g is sent to the input buffer of expert e at buffer position c. For a specific `g` and `s`, a slice `combine_weight[g, s, :, :]` contains at most two non-zero vaules. Binary `dispatch_mask` is produced from `combine_weights` by simply setting all non-zero values to 1.

Algorithm 2: Forward pass of the Positions-wise MoE layer. The underscored letter (e.g., G and E) indicates the dimension along which a tensor will be partitioned.

```
gates = softmax(einsum("GSM,ME->GSE", inputs, wg))
combine_weights, dispatch_mask = Top2Gating(gates)
dispatched_expert_inputs = einsum(
    "GSEC,GSM->EGCM", dispatch_mask, reshaped_inputs)
h = einsum("EGCM,EMH->EGCH", dispatched_expert_inputs, wi)
h = relu(h)
expert_outputs = einsum("EGCH,EHM->GECM", h, wo)
outputs = einsum(
    "GSEC,GECM->GSM", combine_weights, expert_outputs)
```



(a) A partitioned `Einsum` operator. Colored letters (G and E) represent the partitioned dimension of each tensor. The partitioner decides to first execute a batch-parallel `Einsum` along the G dimension, then reshard the result to the E dimension.

User annotation

- **replicate(tensor)** annotates `tensor` to be replicated across partitions, and returns the annotated tensor. This is often used for the non-MoE layers in our model to replicate the weights.
- **split(tensor, split_dimension, num_partitions)** annotates `tensor` to be partitioned along `split_dimension`, and returns the annotated tensor. Partition i is placed on the i'th device, and `num_partitions` must not exceed the number of devices on the system.
- **shard(tensor, device_assignment)** generalizes `split()` to allow partitioning multiple dimensions and specifying the placement of each partition. Appendix A.3 describes this API with more details.

MoE parallelization

```
  # Partition inputs along group (G) dim.
+ inputs = split(inputs, 0, D)
  # Replicate the gating weights
+ wg = replicate(wg)
  gates = softmax(einsum("GSM,ME->GSE", inputs, wg))
  combine_weights, dispatch_mask = Top2Gating(gating_logits)
  dispatched_expert_inputs = einsum(
    "GSEC,GSM->EGCM", dispatch_mask, reshaped_inputs)
  # Partition dispatched inputs along expert (E) dim.
+ dispatched_expert_inputs = split(dispatched_expert_inputs, 0, D)
  h = einsum("EGCM,EMH->EGCH", dispatched_expert_inputs, wi)
  ...
```

DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models

Damai Dai*[1,2], Chengqi Deng[1], Chenggang Zhao*[1,3], R.X. Xu[1], Huazuo Gao[1], Deli Chen[1], Jiashi Li[1], Wangding Zeng[1], Xingkai Yu*[1,4], Y. Wu[1], Zhenda Xie[1], Y.K. Li[1], Panpan Huang[1], Fuli Luo[1], Chong Ruan[1], Zhifang Sui[2], Wenfeng Liang[1]

[1]**DeepSeek-AI**
[2]**National Key Laboratory for Multimedia Information Processing, Peking University**
[3]**Institute for Interdisciplinary Information Sciences, Tsinghua University**
[4]**National Key Laboratory for Novel Software Technology, Nanjing University**
{daidamai, szf}@pku.edu.cn, {wenfeng.liang}@deepseek.com
https://github.com/deepseek-ai/DeepSeek-MoE

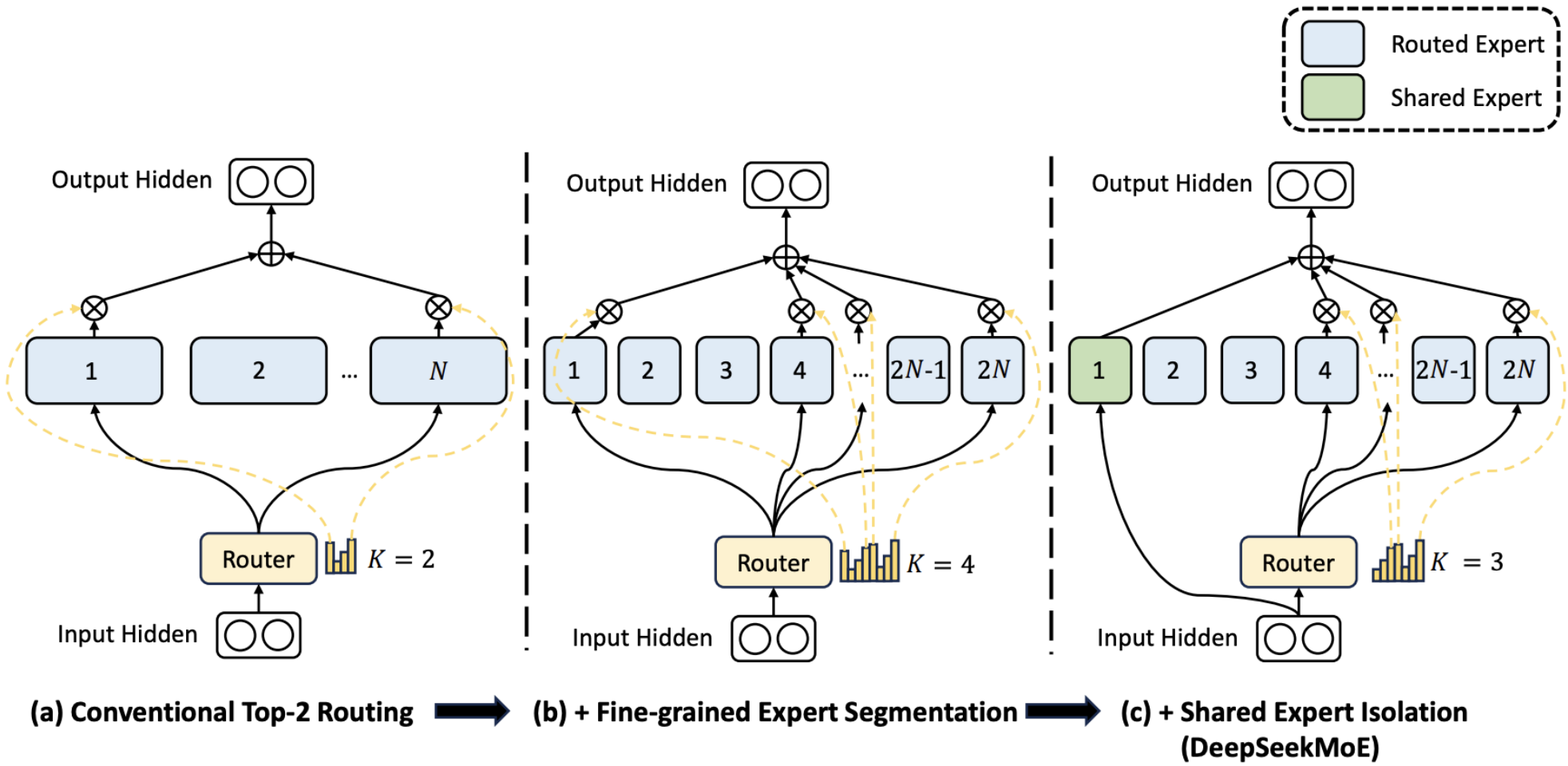


Figure 2 | Illustration of DeepSeekMoE. Subfigure (a) showcases an MoE layer with the conventional top-2 routing strategy. Subfigure (b) illustrates the fine-grained expert segmentation strategy. Subsequently, subfigure (c) demonstrates the integration of the shared expert isolation strategy, constituting the complete DeepSeekMoE architecture. It is noteworthy that across these three architectures, the number of expert parameters and computational costs remain constant.

Starting from a modest scale with 2B parameters, we validate the advantages of the DeepSeek-MoE architecture. We conduct evaluations on 12 zero-shot or few-shot benchmarks spanning diverse tasks. Empirical results indicate that DeepSeekMoE 2B surpasses GShard 2B (Lepikhin et al., 2021) by a substantial margin, and even matches GShard 2.9B, a larger MoE model with 1.5× expert parameters and computation. Remarkably, we find that DeepSeekMoE 2B nearly approaches the performance of its dense counterpart with an equivalent number of parameters, which sets the strict upper bound of MoE language models. In pursuit of deeper insights, we conduct elaborate ablation studies and analysis on the expert specialization for DeepSeekMoE. These studies validate the effectiveness of fine-grained expert segmentation and shared expert isolation, and provide empirical evidence supporting the assertion that DeepSeekMoE can achieve a high level of expert specialization.

Leveraging our architecture, we subsequently scale up the model parameters to 16B and train DeepSeekMoE 16B on a large-scale corpus with 2T tokens. Evaluation results reveal that with only about 40% of computations, DeepSeekMoE 16B achieves comparable performance with DeepSeek 7B (DeepSeek-AI, 2024), a dense model trained on the same 2T corpus. We

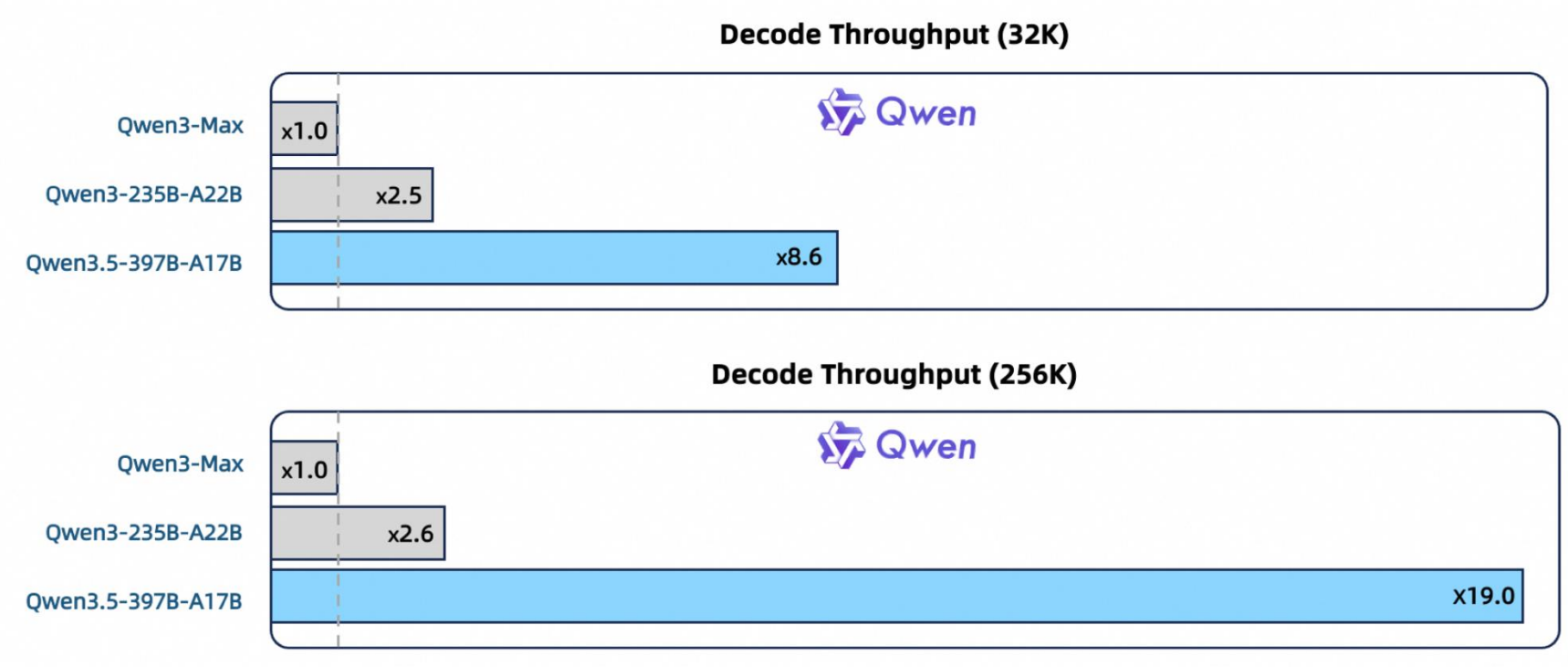
DeepSeekMoE 16B consistently outperforms models with a similar number of activated parameters by a large margin, and achieves comparable performance with LLaMA2 7B (Touvron et al., 2023b), which has approximately 2.5 times the activated parameters. Figure 1 demonstrates

the evaluation results on the Open LLM Leaderboard[2]. Additionally, we conduct supervised fine-tuning (SFT) for alignment, transforming the model into a chat model. Evaluation results show that DeepSeekMoE Chat 16B also achieves comparable performance with DeepSeek Chat 7B and LLaMA2 SFT 7B in the chat setting. Encouraged by these results, we further undertake a preliminary endeavor to scale up DeepSeekMoE to 145B. The experimental results still validate its substantial advantages over the GShard architecture consistently. In addition, it shows performance comparable with DeepSeek 67B, using only 28.5% (maybe even 18.2%) of computations.

Qwen3.5-397B-A17B

Qwen3.5

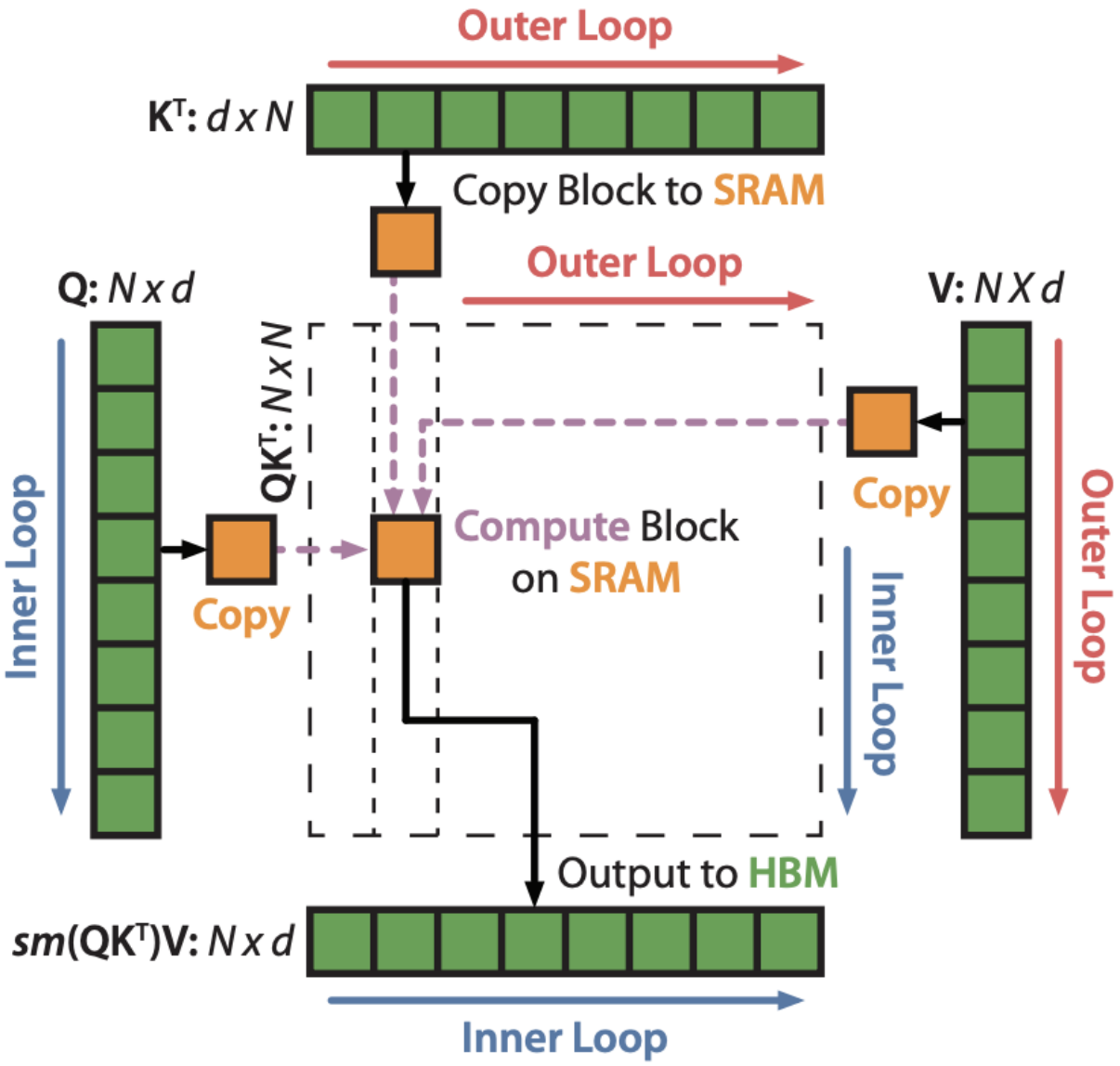
- Mixture Of Experts
  - Number of Experts: 512
  - Number of Activated Experts: 10 Routed + 1 Shared
  - Expert Intermediate Dimension: 1024

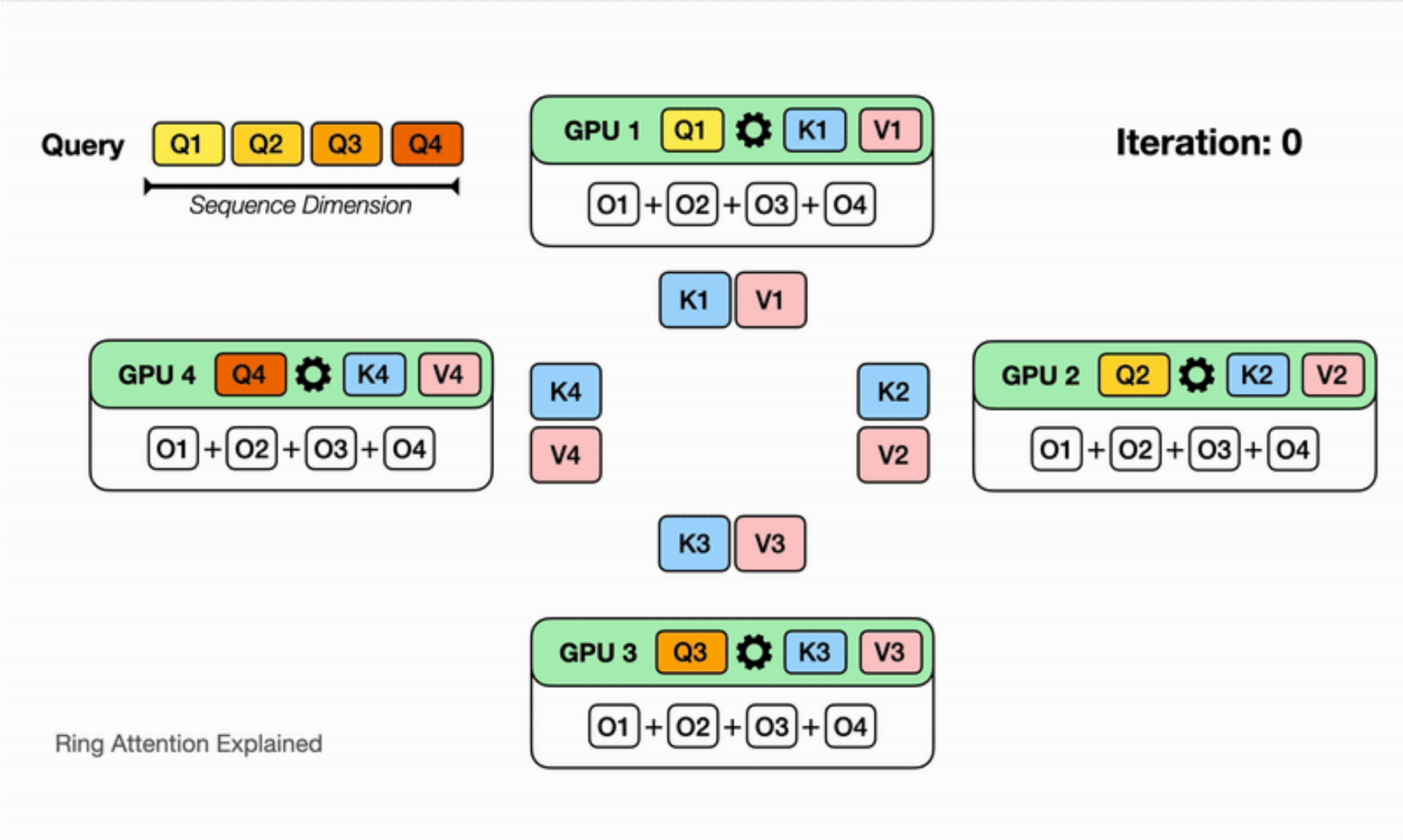


Context Parallelism

- We need to split the sequence dimension for training

- But how do we compute attention?
  - We partition q using the sequence dimension. Every GPU computes a subsequence of q.
  - In order to compute attention, every GPU needs the whole k and v.
    - Naively, we can do allgather to get k and v



FlashAttention

Ring Attention